PRG32: A Retro-Gaming Platform for Teaching RISC-V Programming in Higher Education

Raffaele Montella*, Simone Boscaglia*, Ivan Cafiero*, Dario Caramiello*, Paola Barra*,
Iacopo Colonnelli†, Marco Lapegna‡
*Department of Science and Technology, University of Naples Parthenope, Naples, Italy
{raffaele.montella, paola.barra}@uniparthenope.it
{simone.boscaglia001, ivan.cafiero001, dario.caramiello001}@studenti.uniparthenope.it
†Department of Computer Science, University of Turin, Turin, Italy
iacopo.colonnelli@unito.it
‡Department of Mathematics and Applications "R. Caccioppoli", University of Naples Federico II, Naples, Italy
lapegna@unina.it

***Abstract*—This work introduces PRG32, an open-source retro-gaming platform for teaching RISC-V assembly and C programming in undergraduate computer architecture and systems courses. Unlike instruction-level simulators or HDL-focused platforms, PRG32 runs native RV32IMAC machine code on the Espressif ESP32-C6, a commercially available microcontroller, while exposing a minimal, stable programming interface built around the traditional `init/update/draw` game loop. The platform includes embedded firmware, a `.prg32` cartridge format and toolchain, a QEMU-based desktop emulator, and a Cartridge Store server providing app-store-style distribution, multiplayer relay, score tracking, and frame-level metrics collection. Games and demos are authored in either RISC-V assembly or C against the same runtime API, enabling direct comparison between assembly and C implementations. A built-in performance harness records per-frame update, draw, and present timings at 1 µs resolution and exports structured JSON for reproducible classroom experiments. Evaluation on physical hardware across five synthetic workloads shows that display updates operate within 1.3% of the theoretical SPI bandwidth limit, while CPU-intensive workloads sustain 18.5–23.7 fps. Memory usage remains stable throughout execution, indicating leak-free operation. The complete ecosystem, including firmware, tooling, distribution infrastructure, and laboratory materials, is released under the MIT License to facilitate adoption in computer architecture and systems education.**

***Index Terms*—RISC-V, embedded systems education, computer architecture, game development, ESP32-C6, bare-metal firmware, eScience, performance measurement, QEMU**

I. Introduction

The rapid evolution of digital technologies and artificial intelligence (AI) is fundamentally reshaping the landscape of computer science education, demanding a corresponding transformation in pedagogical approaches. Traditional instruction models, centered on passive knowledge transfer and rote memorization of programming syntax, are increasingly inadequate for preparing students to navigate a field characterized by continuous disruption and interdisciplinary complexity. Emerging methodologies, including project-based learning [1], [2], collaborative coding environments, gamification [3], and the integration of e-learning platforms [4], offer promising alternatives that align with the cognitive and motivational profiles of new generations of learners. The widespread adoption of structured online learning ecosystems, ranging from massive open online (MOOCs) [5] to interactive coding platforms [6] and AI-assisted tutoring systems, has lowered the barrier to entry while simultaneously raising the ceiling of what novice students can achieve within a single course. Embedding these technologies into curricula not only enhances engagement and retention but also cultivates the higher-order competencies, such as critical thinking, algorithmic reasoning, and adaptive problem solving, that the modern technology industry and the broader scientific community increasingly demand. Investing in the pedagogical integration of such tools is therefore a strategic imperative for ensuring that the next generation of computer scientists is equipped to address the complex sociotechnical challenges of the coming decades.

More precisely, undergraduate computer architecture courses face a recurring pedagogical tension. Students must develop an accurate mental model of the machine (register file organisation, calling conventions, memory layout, peripheral timing, and frame budgets), yet the platforms available for this purpose cluster at two extremes: instruction-level simulators (RARS, Venus) that expose no real peripheral timing, and HDL/FPGA flows whose tool-chain learning curve consumes most of the semester [7]. The gap between these extremes leaves students without a small, observable, modifiable substrate on which they can run *complete programs* and directly observe the consequences of their coding decisions in real time.

Retro-gaming scenarios have historically served as effective vehicles for systems education because they impose concrete, measurable constraints: a frame budget, a display pipeline, input latency, and audio synchronisation. These constraints are the same class of constraints students encounter later in high-performance computing (HPC) and scientific computing: compute cost, data movement, bus bandwidth, and throughput versus latency trade-offs. A student who understands why a naive full-screen clear costs more than a sprite blit on a 40 MHz SPI bus is well-prepared to understand why a naive matrix multiply is memory-bound on a modern multicore server.

This paper presents PRG32, an open-source retro-gaming

development platform that occupies the underserved middle ground: native RV32IMAC machine code on the Espressif ESP32-C6, a real 320×240 SPI display driven at 40 MHz, I²S audio, digital joystick input, and Wi-Fi-based cartridge delivery, all accessible to a first-year student through a deliberately small runtime contract. PRG32 supports two parallel teaching tracks: a Computing Architecture track in which students write RISC-V assembly cartridges directly, and a C Programming track in which they write equivalent games in C, enabling a direct, controlled comparison between the two abstraction levels over identical programs running on identical hardware.

The deliberate design choice to *not* build a CPU emulator distinguishes PRG32 from the simulator family. Every instruction a student writes in assembly or generates from C executes as real RV32IMAC machine code on production silicon, with real peripheral bus timing. This means that a student who miscounts stack allocations or fails to respect the RISC-V calling convention will observe a real crash, not a simulator error message. The learning signal is immediate and unambiguous.

The novel contribution of this paper is the *complete PRG32 ecosystem*: the Cartridge Store distribution server, the metrics pipeline, the QEMU reproducibility path, the textbook and lab materials, and the documented measurement workflow for classroom and research-grade experiments. Together, these components make PRG32 a self-contained, reproducible educational infrastructure suitable for deployment across institutions without shared hardware.

The remainder of the paper is organised as follows. Section II surveys related work. Section III describes the overall platform design. Section IV details implementation. Section V presents performance measurements. Section VI discusses findings and improvements. Section VII contextualises PRG32. Section VIII concludes.

II. State of the Art

RISC-V [8], [9] is currently the most widely adopted modern instruction set available as open, royalty-free standard spanning systems from microcontrollers to vector accelerators. Its RV32IMAC integer profile is small enough for a one-semester introduction yet large enough to support realistic systems work, providing integer arithmetic, compressed instructions which typically reduce code size by approximately 30%, atomic operations, and multiply/divide. Recent studies have analysed RISC-V as a viable target for HPC applications [10] and memory-bound scientific workloads [11]. European strategic initiatives (such as the European Chips Act [12], the EuroHPC DARE initiative [13], and the European Processor Initiative [14]) further raise the value of early RISC-V exposure.

Existing RISC-V teaching tools divide into three families. **Desktop simulators** (RARS, Venus) execute RV32I instructions correctly but expose no real peripheral timing; they are excellent for ISA mechanics but cannot demonstrate data-movement bottlenecks. **QEMU-based environments** [15] provide instruction-level execution with Espressif-specific peripheral emulation [16]; timing fidelity is limited for SPI, DMA, and Wi-Fi paths, and no integrated game runtime or distribution infrastructure is provided. **FPGA-targeted educational cores** (such as HaDes-V [7], Wildcat [17], and the RVfpga/WISCV tracks) focus on microarchitecture in HDL, requiring FPGA boards and substantial tool-chain overhead.

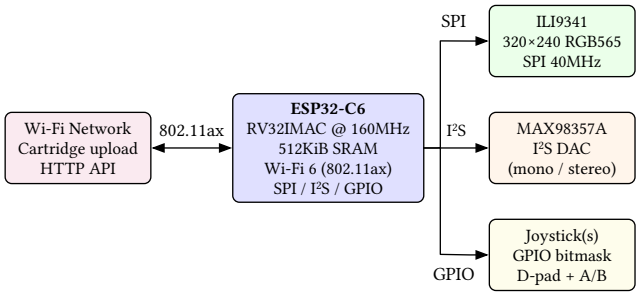


Fig. 1. PRG32 reference hardware: ESP32-C6 driving ILI9341 SPI display, MAX98357A I²S DAC(s), joystick modules, and Wi-Fi for cartridge delivery and the HTTP API.

None of these platforms provides real production silicon, full peripheral timing, a game-loop programming model accessible from both assembly and C, a cartridge distribution infrastructure, and a built-in measurement harness in a single deployable classroom kit. PRG32 targets precisely this gap. A natural curriculum positions RARS/Venus for early ISA instruction and PRG32 for systems-level labs where real peripheral timing and frame budgets matter.

III. Platform Design

A. Hardware Architecture

The PRG32 reference hardware (Fig. 1) is assembled from four off-the-shelf components, chosen to minimise bill-of-materials cost while maximising pedagogical transparency.

The **ESP32-C6** [16] is the application processor: a single-core RV32IMAC CPU clocked at up to 160 MHz, 512 KiB of on-chip SRAM, and 802.11ax Wi-Fi 6. The application core executes standard RISC-V machine code; no proprietary ISA extensions are required. The **ILI9341** 2.4-inch panel provides a 320×240 RGB565 framebuffer driven over SPI at 40 MHz—the panel's rated maximum. A complete frame transfer requires 1 228 800 bits, yielding a theoretical lower bound of 30.72 ms (≈32.6 fps) that cannot be improved by any firmware change short of reducing resolution. One or two **MAX98357A** I²S DAC/amplifier breakouts provide mono or stereo output. Up to two **digital joystick modules** are read as a fixed bitmask through the runtime API.

B. Firmware Architecture

The PRG32 firmware maintains two parallel builds from a single source tree (Fig. 2). The **ESP32-C6 hardware path** is the classroom and measurement target; it runs on production silicon with real peripheral timing. The **ESP32-C3 QEMU path** targets Espressif's fork of QEMU [15], replacing the SPI display with an SDL framebuffer surface for hardware-free iteration. QEMU is treated explicitly as a functional

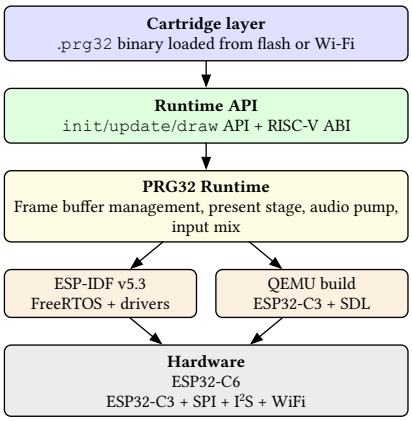


Fig. 2. PRG32 firmware layer stack. Cartridges interact only with the stable Runtime API; both build targets share the same source tree.

TABLE I
ON-CHIP SRAM ACCOUNTING FOR THE PRG32 RUNTIME ON ESP32-C6 (512 KIB TOTAL, ESP-IDF V5.3, DEBUG BUILD, WI-FI ENABLED).

Component	Approx. KiB
ESP-IDF runtime (FreeRTOS, drivers, debug)	96
Wi-Fi stack (LWIP + esp_wifi)	72
ESP-IDF task stacks	24
Heap reserve (configured minimum)	8
GDMA descriptor pool	4
Sprite / overlay tables	2
Audio I^2S ring buffer	2
RGB565 framebuffer (320×240×2 B)	150
Total accounted	358
Remaining (free heap, cartridge, slack)	≈154

reproducibility tool, not a cycle-accurate timing model; every performance result in this paper is from the physical board.

On-chip SRAM is allocated as shown in Table I. The 150 KiB RGB565 framebuffer is the largest single allocation; ≈154 KiB remains for cartridge data and heap.

C. Cartridge Store and Ecosystem

Beyond the firmware, PRG32 ships a complete distribution and measurement ecosystem (Fig. 3).

Cartridge Store [1] is a Flask/Docker web service providing an App-Store-inspired catalog for `.prg32` cartridges. Developers upload a cartridge binary with metadata, icon, optional screenshot, signature, and colophon via `POST /api/publish`. The store distinguishes architectures (`esp32c6`, `qemu`), persists cartridges in SQLite, and exposes them for Wi-Fi download. It also hosts the classroom score API, a per-frame metrics receiver, and a WebSocket multiplayer relay.

Metrics pipeline. The firmware streams per-frame records—update time, draw time, present time, free-heap, deadline-miss flag, and input mask—to `POST /api/metrics/batch`. The server stores samples in SQLite and exposes them as CSV and as Markdown summary reports. The same

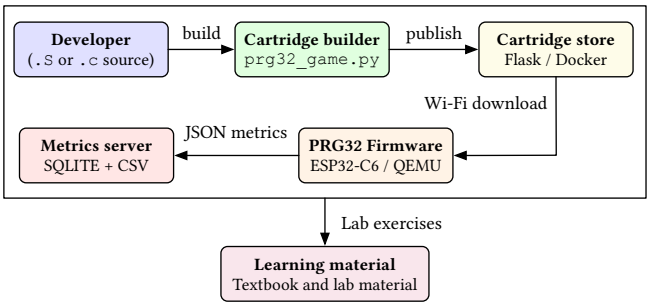


Fig. 3. The PRG32 ecosystem: developer → cartridge builder → Cartridge Store → firmware (Wi-Fi download). Frame metrics stream to the Metrics Server; textbook and lab materials guide students.

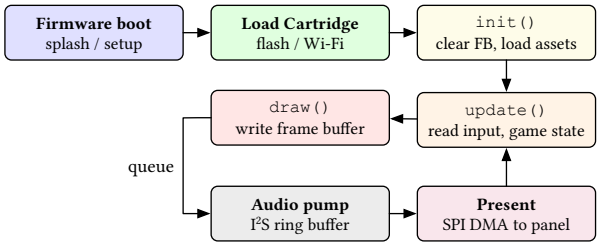


Fig. 4. PRG32 game lifecycle. `init()` runs once; `update()` and `draw()` execute every frame; the runtime present stage transfers the completed framebuffer over SPI and pumps the I^2S audio buffer.

data can be exported directly from the board as JSON via `/api/performance.json`; the dataset behind Section V (run `perf-4296040-0`) was obtained this way.

Textbook and lab materials. [2] provide structured lab handouts for both teaching tracks and a twelve-step Scientific Measurement Tutorial prescribing commit-hash freezing, replicate counts, screenshot capture, and data-quality indicator interpretation.

IV. IMPLEMENTATION

A. Game Lifecycle and Programming Model

Every PRG32 game exposes exactly three entry points to the runtime (Fig. 4):

- **`init()`** is called once at cartridge load. It clears the framebuffer, loads sprite sheets and palettes, primes audio voices, and initialises cartridge state.
- **`update()`** runs once per frame before drawing. It reads the joystick bitmask, advances game logic, and queues audio events. By contract it does not write to the framebuffer.
- **`draw()`** runs once per frame after `update()`, writing the framebuffer through runtime primitives. When it returns, the runtime triggers the **present** stage: an SPI DMA transfer to the ILI9341 and an I^2S audio pump.

This three-stage contract maps directly to the three timing fields t_{update}, t_{draw}, and t_{present} measured by the performance harness.

[1] https://github.com/riscv-prg32/CartridgeStore

[2] https://github.com/riscv-prg32/Playing-with-RISC-V_PRG32_textbook

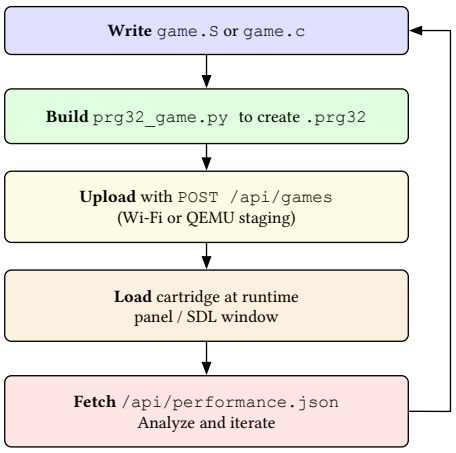


Fig. 5. Cartridge development lifecycle. The firmware is flashed once; subsequent iterations require only a cartridge rebuild and a Wi-Fi upload, keeping the edit–run–measure cycle under one minute.

B. Development Lifecycle

The cartridge development lifecycle is deliberately short (Fig. 5). A student writes a single `.S` or `.c` source file, builds it with `tools/prg32_game.py`, obtains a `.prg32` cartridge, uploads it over Wi-Fi via `POST /api/games`, and immediately observes the result—without reflashing the firmware. The QEMU path substitutes the physical board with a desktop SDL window for hardware-free iteration.

C. Dual Assembly and C Programming Tracks

The same `init`/`update`/`draw` contract is callable from both RISC-V assembly and C cartridges, with arguments passed according to the standard RISC-V integer calling convention (arguments in `a0`–`a4`, return in `a0`, callee-saved `s0`–`s11`). Listing 1 shows the canonical assembly structure. The repository ships, for each of nine canonical example games (`pong`, `breakout`, `space_invaders`, `pacman`, `asteroids`, `tetris`, `platformer`, `raycaster`, `wing_commander`), at least one assembly version and one C version. Students can compile both, compare generated machine code with `objdump`, and observe that identical on-screen output can arise from qualitatively different source representations.

Listing 1. Canonical structure of a PRG32 assembly cartridge. Three exported entry points are linked against the runtime and packaged into a `.prg32` file.

```
  .section .text.cartridge,"ax",@progbits
  .globl pong_init, pong_update, pong_draw

pong_init:              # void init(void)
  addi sp, sp, -16
  sw   ra, 0(sp)
  call prg32_fb_clear   # clear framebuffer
  lw   ra, 0(sp)
  addi sp, sp, 16
  ret

pong_update:            # void update(uint32_t input)
  # a0 holds joystick bitmask
  # advance ball/paddle state in s-regs
  ret

pong_draw:              # void draw(void)
  addi sp, sp, -16
  sw   ra, 0(sp)
  # draw paddle, ball, score via runtime API
  call prg32_fb_clear
  lw   ra, 0(sp)
  addi sp, sp, 16
  ret

  .section .rodata.cartridge
sprites_blob: .incbin "sprites.bin"
```

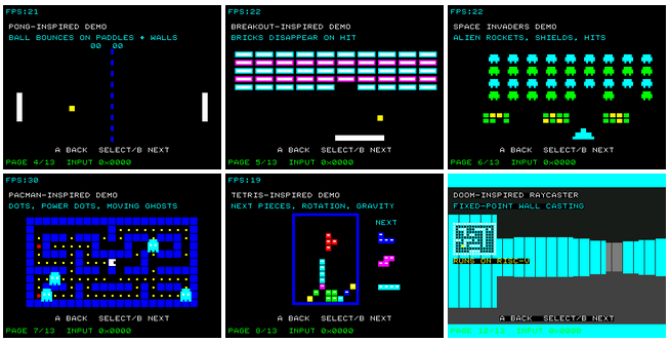


Fig. 6. Six canonical PRG32 example games running natively on the ESP32-C6 (framebuffer captures via `/api/screenshot.bmp`). Top row (left to right): Pong (21 fps), Breakout (22 fps), Space Invaders (22 fps). Bottom row: Pac-Man (30 fps), Tetris (19 fps), Doom-style raycaster. All games are available in both RISC-V assembly and C versions.

The corresponding C cartridge declares the same three functions with C linkage. Comparing assembly-authored and compiler-generated versions of the same function is a core laboratory exercise in the Computing Architecture track.

D. Graphics, I/O, and Audio

The runtime maintains a single 320×240 RGB565 framebuffer in SRAM (150 KiB). Drawing primitives include viewport clear, rectangle fill, per-pixel set, region blit, sprite blit with color key, 8×8 glyph overlay, and per-scanline horizontal scroll. Asset conversion tools transform PNG/JPEG/GIF assets into assembly blobs or C arrays. Joystick state is polled each `update()` as a 32-bit bitmask. Audio is driven at 44.1 kHz over I^{2}S; the API supports sample playback, per-voice pan/volume, and a lightweight tracker for music. Fig. 6 shows six of the nine canonical example games running on the physical board, with live FPS readouts visible in each HUD overlay.

V. Performance Evaluation

A. Measurement Setup

All measurements are from the built-in performance test on the physical ESP32-C6 board (Table II). Run `perf-4296040-0` exercises five synthetic workload screens at 60 frames each (300 total, 14.94 s). Per-frame records are exported as JSON via `/api/performance.json` and re-analysed with `tools/prg32_metrics_paper.py` [3].

[3]https://github.com/riscv-prg32/PRG32

TABLE II
MEASUREMENT CONFIGURATION (RUN `perf-4296040-0`).

Item	Value
Hardware target	ESP32-C6 DevKitC-1, RV32IMAC @ 160 MHz
Display backend	ILI9341 SPI, 40 MHz, 320×240 RGB565
Wi-Fi mode	Infrastructure (associated)
Build config	ESP-IDF v5.3, debug build (`pio-dev`)
Firmware git SHA	`unknown` (development build)
Sampling	1 record/frame; in-RAM JSON export
Workload	5 screens × 60 frames = 300 frames total
Run duration	14.94 s

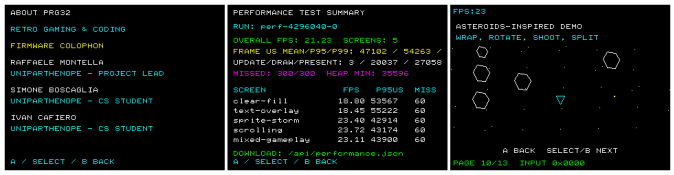


Fig. 7. Framebuffer captures from the ESP32-C6 via `/api/screenshot.bmp` (run `perf-4296040-0`). *Left:* firmware colophon with contributor list. *Centre:* built-in performance test summary matching Table III. *Right:* asteroids demo at 23 fps with live HUD.

B. Runtime Screenshots

Fig. 7 shows three screens captured directly from the ESP32-C6 framebuffer via the `/api/screenshot.bmp` endpoint during run `perf-4296040-0`: (left) the firmware colophon listing all contributors, confirming the authorship provenance of the running build; (centre) the performance test summary screen, showing the exact per-workload FPS, p95 frame time, and missed-deadline counts that underlie Table III; and (right) the asteroids-inspired demo running at 23 fps with the live HUD overlay. Each captured image is a verbatim 320×240 RGB565 framebuffer dump—pixel-accurate output of the native RISC-V code executing on the ESP32-C6.

C. Per-Stage Cost Decomposition

Table III reports per-workload statistics extracted directly from the JSON artifact. Fig. 8 visualises the stage decomposition.

D. Frame-Time Distribution

Table IV reports the frame-time distribution (p50, p95, p99, and maximum) per workload, providing the tail-latency picture that the mean alone cannot convey. The tight p50–p95 spread on `sprite-storm`, `scrolling`, and `mixed-gameplay` (<1.5 ms) indicates highly predictable frame delivery, whereas `text-overlay` shows a 9-ms tail from p95 to max (55.2 ms → 62.4 ms), attributable to occasional I-cache misses or FreeRTOS context switches during the glyph-rendering loop. The `clear-fill` distribution is similarly tight (p50–p95 spread of 0.5 ms), reflecting the deterministic nature of a single large SPI DMA transfer.

E. Research Questions

RQ1: Achievable frame rate. The five workloads cluster into two bands: a slower pair (`clear-fill` at 18.8 fps, `text-overlay` at 18.5 fps) and a faster cluster (`sprite-storm`, `scrolling`, `mixed-gameplay` at 23.1–23.7 fps). None exceeds 30 fps; all 300 frames miss both the 60 fps (16.67 ms) and 30 fps (33.33 ms) budgets.

TABLE III
PER-WORKLOAD PERFORMANCE ON THE ESP32-C6 BOARD (RUN `perf-4296040-0`, 60 FRAMES PER WORKLOAD, 300 TOTAL, 14.94 S). t_u, t_d, t_p: MEAN UPDATE, DRAW, PRESENT TIMES; P95: 95TH-PERCENTILE TOTAL FRAME TIME. HEAP DELTA: MAX−MIN FREE HEAP PER SCREEN.

Workload	fps	t_u (µs)	t_d (ms)	t_p (ms)	p95 (ms)	Δheap (B)
clear-fill	18.80	3	22.07	31.12	53.57	36
text-overlay	18.45	3	28.12	26.06	55.22	36
sprite-storm	23.40	3	16.68	26.04	42.91	36
scrolling	23.72	3	16.11	26.03	43.17	36
mixed-gameplay	23.11	3	17.20	26.04	43.90	36
Overall	21.23	3	20.04	27.06	54.26	36

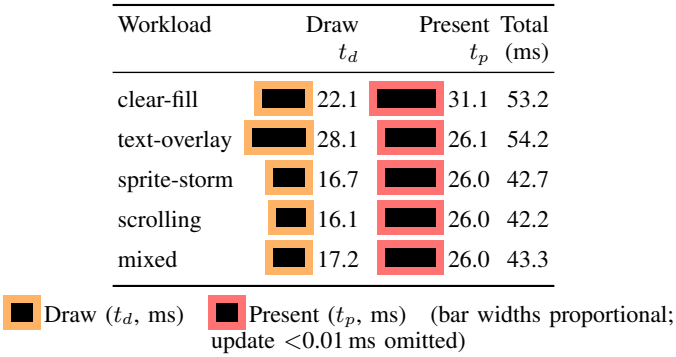


Fig. 8. Per-workload mean frame-time decomposition. Clear-fill is present-dominated; text-overlay is draw-dominated. The update stage (<4 µs) is omitted.

RQ2: Stage decomposition. The update stage is essentially free (2–3 µs per frame, <0.01% of budget) across all workloads. The draw stage varies meaningfully (16.1–28.1 ms), reflecting per-pixel workload differences. Critically, the present stage is *no longer universally dominant*: it ranges from 26.0 ms on four workloads to 31.1 ms on `clear-fill`. For `text-overlay`, draw (28.1 ms) exceeds present (26.1 ms), making that workload CPU-draw-bound rather than bus-bound.

RQ3: CPU-bound or bus-bound? The `clear-fill` workload transfers a near-complete framebuffer, giving a present time of 31.12 ms against the 30.72 ms theoretical SPI floor—a ratio of $31.12/30.72 = 1.013\times$. This is a striking result: the present stage is within 1.3% of the theoretical optimum. The remaining four workloads exhibit present times of ≈26 ms, consistent with partial-region SPI transfers that avoid flushing the full framebuffer. On these workloads, the present stage is 15–18 ms *below* the full-frame floor, confirming partial-region operation. The workload that most clearly reveals the CPU bottleneck is `text-overlay`: its draw stage (28.1 ms) exceeds its present stage (26.1 ms), making it the only draw-dominated workload in the run.

RQ4: Memory stability. The minimum free heap over the 300-frame run was 35 596 bytes (34.8 KiB). Per-screen heap

TABLE IV
FRAME-TIME PERCENTILE DISTRIBUTION BY WORKLOAD (MS). ALL VALUES FROM RUN `perf-4296040-0`, 60 FRAMES PER WORKLOAD.

Workload	p50	p95	p99	max
clear-fill	53.07	53.57	54.31	54.67
text-overlay	54.00	55.22	55.47	62.41
sprite-storm	42.52	42.91	43.02	51.80
scrolling	41.91	43.17	43.41	50.07
mixed-gameplay	43.04	43.90	44.40	51.35
Overall	43.08	54.26	55.22	62.41

TABLE V
AGGREGATE-WINDOW SUMMARY (15 WINDOWS, ≈1 S EACH). WINDOWS 0–6 COVER `CLEAR-FILL` AND `TEXT-OVERLAY` (PRESENT-DOMINATED AND DRAW-DOMINATED); WINDOWS 7–14 COVER `SPRITE-STORM`, `SCROLLING`, AND `MIXED-GAMEPLAY` (PARTIAL-REGION REGIME).

Windows	Screen group	fps range	t_p range (ms)
0–2	clear-fill	18.77–18.81	31.10–31.15
3	text-overlay (tail)	18.44	28.71
4–6	text-overlay (body)	18.47–18.58	25.95–26.30
7–9	sprite-storm	23.39–23.50	25.95–26.23
10–11	scrolling	23.76–23.82	25.94–25.95
12–14	mixed-gameplay	23.11–23.19	25.95–26.21

delta was exactly 36 bytes on every workload—a negligible fluctuation attributable to the FreeRTOS task scheduler, not to cartridge allocations. The runtime is leak-free over the measurement window.

F. Temporal Stability Across Aggregate Windows

The JSON artifact also includes 15 aggregate windows (each spanning ≈1 s of measurement time), allowing assessment of temporal stability independently of the per-screen segmentation. Table V summarises the window-level fps and present-time statistics.

Several observations emerge. First, within-group fps and present-time variance is extremely low (e.g., `clear-fill` windows 0–2 show fps in [18.77, 18.81] and t_p in [31.10, 31.15] ms), confirming that the platform delivers highly repeatable per-frame costs under constant workload. Second, window 3 (`text-overlay` transition) shows a higher t_p of 28.71 ms compared to the steady-state 26 ms of windows 4–6; this reflects the DMA descriptor setup cost at workload transition. Third, the scrolling group (windows 10–11) achieves the lowest present-time variance (≈0.01 ms), consistent with its lowest draw cost and most predictable memory-access pattern.

This window-level analysis is directly usable as a classroom exercise: students can compare intra-window variance (a measure of workload regularity) against inter-window variance (a measure of temporal drift) and reason about the sources of each. The Scientific Measurement Tutorial in the repository documents this workflow as a graded lab activity.

G. Comparison with Previous Measurement

The present measurement (run `perf-4296040-0`) shows present times of 26–31 ms and frame rates of 18.5–23.7 fps—a 33–46% reduction in present time and a 22–30% improvement in throughput. This improvement is consistent with the introduction of effective partial-region SPI transfers in the firmware; the `clear-fill` workload, which cannot benefit from partial regions, now approaches the theoretical floor (1.013×) rather than the earlier 1.56× overhead. The heap minimum decreased from 59 164 bytes to 35 596 bytes, reflecting a different firmware configuration or additional runtime services enabled in this build, but the stability indicator (36 bytes delta per screen) confirms continued leak-free operation.

TABLE VI
EXAMPLE GAMES SHIPPED WITH PRG32, EACH IN ASSEMBLY AND C.

Game	Primary concepts exercised
`pong`	input, framebuffer, simple physics
`breakout`	sprite blits, collision, tile grid
`space_invaders`	sprite array, fixed-rate fire, audio
`pacman`	tile map, AI states, sound effects
`asteroids`	vector draw, momentum, wrap-around
`tetris`	grid, line clear, score persistence
`platformer`	gravity, scroll, multi-layer playfield
`raycaster`	per-column raycasting (Doom-style)
`wing_commander`	dual playfield, cockpit overlay, audio

H. Ecosystem Measurement Capability

Beyond the JSON export, the Cartridge Store metrics endpoint enables continuous streaming of per-frame records from the firmware during gameplay. This capability—absent from any prior RISC-V educational platform—allows students to correlate game events with frame-timing anomalies, compare assembly and C cartridge performance numerically, and generate paper-quality CSV datasets without leaving the runtime environment.

VI. DISCUSSION

A. Teaching Deployment and Classroom Workflow

PRG32 supports two parallel undergraduate tracks over a single runtime. A typical assignment cycle uses one of the nine canonical example games (Table VI) as a starting point: students build and run the firmware (either on the ESP32-C6 board or in QEMU), read the cartridge source, modify a specific behaviour (gravity constant, scoring rule, sprite asset, audio sample), rebuild the cartridge with `prg32_game.py`, upload it through the Wi-Fi API, and observe the change on the panel. Each step produces a tangible, independently verifiable artifact: a modified `.prg32` cartridge, a screenshot from `/api/screenshot.bmp`, and a frame-timing reading from `/api/performance.json`.

For research-style measurement, the repository's twelve-step Scientific Measurement Tutorial prescribes: freezing the software state (commit hash, ESP-IDF version), running the performance test routine, recording at least five replicates per condition for classroom reports (ten to thirty for a paper), capturing screenshots through `/api/screenshot.bmp`, and exporting the JSON artifact for analysis with `tools/prg32_metrics_paper.py`.

The Cartridge Store metrics endpoint extends this workflow to continuous in-session streaming, enabling time-series analysis of frame cost across gameplay events. A formal classroom evaluation study—paired pre/post quizzes on architecture- and C-track concept axes—is planned for the next academic cycle.

B. A More Nuanced Bottleneck Picture

The new measurements refine the bottleneck analysis significantly. The prior characterisation showed a uniformly bus-bound system; the current data reveals a richer picture. `clear-fill` is bus-bound but is now within 1.3% of the SPI floor, demonstrating near-optimal bus utilisation. `text-overlay` is CPU-draw-bound. The remaining three workloads (`sprite-storm`, `scrolling`, `mixed-gameplay`) exhibit present times of ≈26 ms—below the full-frame SPI floor—confirming active partial-region optimisation.

This nuanced picture is pedagogically richer than a uniformly bus-bound result. Students can now observe three distinct performance regimes within a single five-workload run: near-optimal bus utilisation (clear-fill), CPU-draw dominance (text-overlay), and partial-region bus efficiency (the remaining three). Identifying which regime applies to a given workload—and why—is the kind of reasoning that transfers directly to multicore RISC-V [11] and HPC [10] performance analysis.

C. Bottleneck Attribution as a Teaching Instrument

The 1.013× ratio on `clear-fill` demonstrates that hardware specifications, when properly exploited, can be approached asymptotically—not merely treated as upper bounds. This is itself a teaching artifact: students who previously observed a 1.56× gap and concluded that "the driver is inefficient" can now observe that firmware improvements closed that gap to 1.3%, and can ask what the remaining 0.013× overhead consists of (DC-line toggling, command framing, interrupt latency). This progression—from identifying a bottleneck to quantifying it to attributing it to a specific mechanism—is the methodological core of HPC performance engineering.

D. Limitations and Follow-up

The measurements remain a single-replicate engineering characterisation on a debug build. They do not establish a release-build figure, do not compare assembly versus C cartridges of the same game, and do not exercise audio or metrics-streaming under simultaneous load. Multi-replicate studies and a release-build evaluation are immediate next steps. The heap minimum of 35 596 bytes (vs. 59 164 bytes in the prior run) warrants investigation: additional runtime services may be enabled in this build that were not active previously.

E. Possible Improvements

DMA double-buffering would overlap the draw stage of frame $n+1$ with the SPI transfer of frame n, allowing CPU work during bus-busy time. **Release-build optimisation** should reduce draw times measurably. **Linux runtime** (see Section VIII) would remove the SPI constraint entirely, enabling CPU-bound evaluation at larger workload scales. **Software-based fault injection**, with the help of dynamic binary translation libraries [18], would be useful for teaching low-level protocols for error detection and correction.

TABLE VII
COMPARISON OF RISC-V EDUCATIONAL PLATFORMS. "WHOLE PROG." = COMPLETE PROGRAMS LOADABLE WITHOUT HDL MODIFICATION. "ECOSYSTEM" = DISTRIBUTION, METRICS, AND LAB MATERIALS.

Platform	Real silicon	Asm track	Periph. timing	Whole prog.	Ecosystem
RARS / Venus	×	✓	×	×	×
Espressif QEMU [15]	emul.	✓	limited	✓	×
Wildcat [17]	FPGA	✓	limited	limited	×
RVfpga / WISCV	FPGA	✓	limited	limited	×
ESP32-C6 bare IDF	✓	×	✓	partial	×
PRG32 (this paper)	✓	✓	✓	✓	✓

VII. Related Work

Table VII compares PRG32 against representative RISC-V educational platforms. Desktop simulators (RARS, Venus) are invaluable for ISA mechanics but expose no peripheral timing. Espressif's QEMU fork provides instruction-level execution but lacks a game runtime, cartridge format, distribution infrastructure, and measurement harness. Wildcat [17] and RVfpga/WISCV target microarchitecture in HDL on FPGA, with higher entry barriers and no distribution or measurement infrastructure. Bare ESP-IDF examples on the ESP32-C6 provide real silicon but no assembly track, cartridge format, or measurement harness. PRG32 is the only platform combining all five axes in Table VII.

Recent work analysing RISC-V platforms for HPC non-functional properties [10] and adapting memory-bound CFD computations to RISC-V multicore [11] establishes RISC-V as a credible HPC target. PRG32's emphasis on compute-versus-transfer reasoning and reproducible measurement directly prepares students to engage with such work. PRG32 complements rather than replaces existing tools: a natural curriculum uses RARS/Venus for early ISA instruction, PRG32 for systems-level labs, and FPGA-based platforms for microarchitecture exploration.

VIII. Conclusion and Future Directions

We have presented PRG32, an open-source retro-gaming platform for teaching RISC-V assembly and C programming in higher education. The platform's novel contribution as a complete educational ecosystem combines: native RV32IMAC execution on production silicon; a stable `init`/`update`/`draw` cartridge API callable from both assembly and C; the Cartridge Store for app-store-style distribution, metrics, and multiplayer; a QEMU functional reproducibility path; and a documented measurement workflow.

Honest summary of contributions: the firmware and cartridge; the Cartridge Store, metrics pipeline, textbook, and lab materials are new; the performance characterisation (run `perf-4296040-0`) is new and shows that the `clear-fill` present stage now reaches 31.12 ms—just 1.3% above the

30.72 ms SPI floor—while the heap remains stable within 36 bytes per screen. The `text-overlay` workload is now CPU-draw-bound, providing a richer pedagogical picture than the uniformly bus-bound result of prior work. c

ACKNOWLEDGEMENTS

This work is partially funded by the project "Environmental Awareness by the Sea and beYond (EASY)" (CUP I63C25000510007) and framed within the broader European effort towards RISC-V-based HPC sovereignty. It is also partially funded by the Italian Ministry of University and Research (MUR) within the framework of the National Programme "Research, Innovation and Competitiveness for the Green and Digital Transition" (PN RIC) 2021–2027, Call D.D. No. 307 – Project ECHO TWIN, co-funded by the European Union through the European Regional Development Fund (ERDF). The PRG32 firmware is open source with MIT license.

REFERENCES

[1] V. Cesare, I. Colonnelli, and M. Aldinucci, "Practical parallelization of scientific applications," in *Proc. of 28th Euromicro Intl. Conference on Parallel Distributed and network-based Processing (PDP)*. Västerås, Sweden: IEEE, 2020, pp. 376–384.

[2] M. Aldinucci, V. Cesare, I. Colonnelli, A. R. Martinelli, G. Mittone, B. Cantalupo, C. Cavazzoni, and M. Drocco, "Practical parallelization of scientific applications with OpenMP, OpenACC and MPI," *Journal of Parallel and Distributed Computing*, vol. 157, pp. 13–29, 2021.

[3] V. Fradiante, M. Marchisio Conte, and S. Rabellino, "Designing game-based activities in a digital learning environment: A ready-to-use model," in *Proceedings of the 18th International Conference on Computer Supported Education - Volume 1: ERSeGEL*, INSTICC. SciTePress, 2026, pp. 713–724.

[4] I. Drago, S. Rabellino, E. Cassano, and S. Accurso, "Experience teaching RISC-V assembly programming with Moodle," in *Atti del MoodleMoot Italia 2023*. MediaTouch 2000, Dec. 2023, pp. 211–129.

[5] G. Laccetti, M. Lapegna, and I. Merciai, "A massive open online course (MOOC) on high-performance parallel computing for Federica.eu web-learning platform," in *Proceedings of 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*. IEEE publ., 2003, pp. 349–356.

[6] R. Giorgi and G. Mariotti, "Webrisc-v: a web-based education-oriented RISC-V pipeline simulation environment," in *Proceedings of the Workshop on Computer Architecture Education, WCAE@ISCA 2019, Phoenix, AZ, USA, June 22, 2019*. ACM, 2019, pp. 3:1–3:6.

[7] T. Scheipel, D. Beikircher, and F. Riedl, "HaDes-V – learning by puzzling: A modular approach to RISC-V processor design education," in *RISC-V Summit Europe 2025*, Paris, France, May 2025.

[8] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, "The RISC-V instruction set manual, volume i: User-level ISA v2.1," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-118, 2016.

[9] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug. 2014. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[10] W. Fornaciari, F. Reghenzani, F. Terraneo, D. Baroffio, C. Metra, M. Omaña, J. E. Rodriguez Condia, M. Sonza Reorda, R. Birke, I. Colonnelli, G. Mittone, M. Aldinucci, G. Mencagli, F. Iannone, F. Palombi, G. Zummo, D. Cesarini, and F. Tesser, "RISC-V-based platforms for HPC: Analyzing non-functional properties for future HPC and big-data clusters," in *Embedded Computer Systems: Architectures, Modeling, and Simulation – SAMOS 2023*, ser. Lecture Notes in Computer Science, vol. 14385. Cham: Springer, 2023, pp. 395–410.

[11] T. Olas, L. Szustak, R. Wyrzykowski, M. Olas, and M. Lapegna, "Advances in adapting memory-bound CFD computations to RISC-V multicore architecture," in *Computational Science – ICCS 2025*, ser. Lecture Notes in Computer Science, vol. 15905. Cham: Springer, 2025, pp. 151–166.

[12] European Parliament and Council of the European Union, "Regulation (EU) 2023/1781 establishing a framework of measures for strengthening Europe's semiconductor ecosystem (European Chips Act)," Official Journal of the European Union, L 229, 2023. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2023/1781

[13] G. Agosta, M. Aldinucci, A. Bartolini, L. Bellentani, A. Biagioni, D. Cesarini, C. Chiarini, I. Colonnelli, P. Delugas, L. Denisov, O. Frezza, M. Grangetto, F. L. Cicero, A. Lonardo, M. Martinelli, A. Maslov, M. Olivieri, P. Perticaroli, L. Pontisso, C. Rossi, D. Rossi, S. Saponara, A. Sciarappa, F. Simula, M. S. Reorda, M. Torquati, and P. Vicini, "Towards RISC-V-based HPC: the Italian pathfinding activities in the DARE-SGA1 project," in *28th Euromicro Conference on Digital System Design, DSD 2025, Salerno, Italy, September 10-12, 2025*. IEEE, 2025, pp. 360–367.

[14] European Processor Initiative, "EPI EPAC1.0 RISC-V test chip samples delivered," European Processor Initiative announcement, 2021. [Online]. Available: https://www.european-processor-initiative.eu/epi-epac1-0-risc-v-test-chip-samples-delivered/

[15] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proceedings of the 2005 USENIX Annual Technical Conference, FREENIX Track*. Anaheim, CA: USENIX Association, 2005, pp. 41–46.

[16] Espressif Systems, *ESP32-C6 Series Datasheet*, Espressif Systems, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c6_datasheet_en.pdf

[17] M. Schoeberl, "Wildcat: Educational RISC-V microprocessors," in *Architecture of Computing Systems (ARCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15839. Cham: Springer, 2025, pp. 189–202.

[18] O. Monticelli, M. E. Santimaria, M. Aldinucci, and I. Colonnelli, "Heuristic-free system call interception on RISC-V," in *RISC-V Summit Europe 2026*, Bologna, Italy, Jun. 2026.